

Analyseklassenmodell – StudyQuest

Titel des Dokuments:

Analyseklassenmodell – StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.3

Datum:

2. März 2026

Author:innen:

- Michael Steer (Scrum Master)
- Giuliana Carrano (Developer)

Betreuer / Prüfer:

Sascha Wanninger

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbeschreibung
1.0	14.11.2025	M. Steer	Erstfassung der Analyseklassenmodell erstellt
1.1	30.01.2026	G. Carrano	Klassenbeschreibungen ergänzt
1.2	24.02.2026	G. Carrano	Finalversion mit vollständigen Attributen und Methoden
1.3	02.03.2026	M. Steer	Traceability-Matrix erweitert, Architekturübersicht ergänzt, NF-Requirements zugeordnet

Distribution List

Rolle	Name	Organisation	Benachrichtigung
Scrum Master	Michael Steer	DHBW Ravensburg	Bei Änderungen
Product Owner	Luke Engehardt	DHBW Ravensburg	Bei Änderungen
Entwickler	Giuliana Carrano	DHBW Ravensburg	Bei Änderungen
Betreuer	Sascha Wanninger	DHBW Ravensburg	Finale Version

1. Übersicht Analyseklassenmodell

Das Analyseklassenmodell basiert auf den Anforderungen aus der Anforderungsanalyse und modelliert die Kerndomänen-Objekte des StudyQuest-Systems. Es folgt dem 3-Schichten-Modell: - **Präsentationsschicht:** UI-Komponenten (nicht in diesem Modell) - **Anwendungsschicht:** Business Logic - **Datenschicht:** Persistenz

2. Klassen und Verantwortlichkeiten

Klasse: User

Verantwortlichkeit: Verwaltung von Benutzerkonten, Authentifizierung und Profildaten

Attribute:

Attribut	Typ	Beschreibung
id	String	Eindeutige Benutzer-ID (UUID)
email	String	E-Mail für Login & Kommunikation
password_hash	String	Gehashtes Passwort (bcrypt/Argon2)
name	String	Anzeigename des Benutzers
avatar	String	Avatar-Emoji oder URL
studiengang	String	Studiengang des Benutzers (z.B. "Informatik")
level	Integer	Aktuelles Level (1-MAX)
total_xp_earned	Integer	Akkumulierte Erfahrungspunkte
xp	Integer	XP im aktuellen Level (0 bis Threshold)
active_quest_id	String	ID der aktuell aktiven Quest (null wenn keine)

Attribut	Typ	Beschreibung
quest_history	String[]	Array von abgeschlossenen Quest-IDs
is_admin	Boolean	Admin-Flagge für UC13-UC15
is_active	Boolean	Benutzer aktiv/inaktiv (UC15)
current_streak	Integer	Aktuelle Tages-Streak (UC12)
best_streak	Integer	Beste erreichte Streak
last_activity	DateTime	Zeitstempel der letzten Aktion
created_at	DateTime	Account-Erstellungsdatum

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
login()	email, password	User null	Authentifiziert Benutzer
register()	email, password, name	User	Erstellt neuen Account
updateProfile()	name, avatar	Boolean	Aktualisiert Profildaten
addXP()	amount	{newLevel, leveledUp}	Fügt XP hinzu, berechnet Level-Up
completeQuest()	questId	Boolean	Markiert Quest als abgeschlossen
setActiveQuest()	questId	Boolean	Setzt aktive Quest
updateStreak()	-	Integer	Berechnet tägliche Streak

Assoziationen: - 1 User : N Quest (abgeschlossene Quests) - 1 User : N Grade (Schulnoten) - 1 User : N Achievement (freigeschaltete Badges) - 1 User : N LearningSession (Sessions)

Klasse: Quest

Verantwortlichkeit: Verwaltung von Lernaufgaben, Schwierigkeitsgrad und XP-Belohnungen

Attribute:

Attribut	Typ	Beschreibung
id	String	Eindeutige Quest-ID
title	String	Titel der Quest
description	String	Detaillierte Beschreibung
difficulty	Enum	easy, medium, hard

Attribut	Typ	Beschreibung
xp_reward	Integer	XP bei Abschluss (50/100/150)
status	Enum	available, active, completed
created_at	DateTime	Erstellungsdatum
updated_at	DateTime	Letzte Änderung

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
getQuestInfo()	-	{title, desc, xp, difficulty}	Gibt Quest-Details zurück
calculateXP()	-	Integer	Berechnet XP basierend auf Schwierigkeit
isAvailable()	-	Boolean	Prüft ob Quest verfügbar ist

Assoziationen: - N Quest : 1 GameRule (XP-Berechnung) - 1 Quest : N LearningSession (abgeschlossene Sessions diesen Quest)

Klasse: LearningSession

Verantwortlichkeit: Audit-Trail für Quest-Abschlüsse mit Zeitenstempel und Zeitmessung

Attribute:

Attribut	Typ	Beschreibung
id	String	Session-ID
user_id	String	Referenz zu User
quest_id	String	Referenz zu Quest
xp_earned	Integer	Verdiente XP (+ Timer-Bonus)
duration_seconds	Integer	Zeitaufwand in Sekunden
timer_bonus_applied	Boolean	Wurde Timer-Bonus angewendet?
completed_at	DateTime	Abschluss-Zeitstempel
started_at	DateTime	Start-Zeitstempel

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
calculateDuration()	-	Integer	Berechnet Dauer in Sekunden

Methode	Parameter	Rückgabe	Beschreibung
<code>applyTimerBonus()</code>	baseXP	Integer	duration_minutes × xp_per_minute_timer (linearer Timer-Bonus)
<code>logSession()</code>	-	Boolean	Speichert Session in DB

Assoziationen: - N LearningSession : 1 User (Sessions eines Users) - N LearningSession : 1 Quest (Sessions für eine Quest)

Klasse: Grade

Verantwortlichkeit: Verwaltung von Schulnoten und Notenstatistiken

Attribute:

Attribut	Typ	Beschreibung
<code>id</code>	String	Noten-ID
<code>user_id</code>	String	Benutzer-Referenz
<code>module_name</code>	String	Modulname (z.B. "Analysis I")
<code>grade_value</code>	Float	Notenwert (1.0-6.0 oder 0-100)
<code>weight</code>	Float	Gewichtung für Durchschnitt
<code>created_at</code>	DateTime	Erfassungsdatum
<code>updated_at</code>	DateTime	Änderungsdatum

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
<code>calculateAverage()</code>	grades[]	Float	Berechnet Durchschnitt
<code>exportToCSV()</code>	-	String	Exportiert als CSV
<code>importFromCSV()</code>	csvData	Grade[]	Importiert aus CSV

Assoziationen: - N Grade : 1 User (Noten eines Users)

Klasse: Achievement

Verantwortlichkeit: Verwaltung von Badges und Freigeschaltungsbedingungen

Attribute:

Attribut	Typ	Beschreibung
id	String	Achievement-ID (z.B. "quest_starter")
title	String	Badge-Name
icon	String	Emoji oder Icon
description	String	Bedingung zur Freischaltung
unlock_condition	String	Logik (z.B. "completedQuests >= 1")
rarity	Enum	common, rare, epic, legendary

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
checkUnlock()	user	Boolean	Prüft ob Bedingung erfüllt
unlockForUser()	userId	Boolean	Schaltet Badge frei
getProgress()	userId	{current, required}	Zeigt Fortschritt

Assoziationen: - N Achievement : N User (Viele zu Viele: Freigeschaltete Badges)

Klasse: Leaderboard

Verantwortlichkeit: Berechnung und Verwaltung von Rankings nach verschiedenen Kriterien

Attribute:

Attribut	Typ	Beschreibung
criterion	Enum	xp, level, quests, streak
rankings	{user_id, rank, value}[]	Ranking-Einträge
last_updated	DateTime	Letztes Update

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
calculateRankings()	users[]	Ranking[]	Berechnet Ränge nach Kriterium
getUserRank()	userId, criterion	Integer	Gibt Rang eines Users zurück
getTopN()	n	User[]	Top N Spieler
updateStreaks()	users[]	Boolean	Aktualisiert Streaks täglich

Assoziationen: - 1 Leaderboard : N User (Rankings aller User)

Klasse: Notification

Verantwortlichkeit: Verwaltung von Benachrichtigungen und Reminderbenachrichtigungen

Attribute:

Attribut	Typ	Beschreibung
id	String	Notification-ID
user_id	String	Empfänger
type	Enum	quest_complete, level_up, achievement, reminder
message	String	Nachrichtentext
is_read	Boolean	Gelesen/ungelesen
created_at	DateTime	Erstellungszeit

Methoden:

Methode	Parameter	Rückgabe	Beschreibung
createNotification()	userId, type, message	Notification	Erstellt neue Benachrichtigung
markAsRead()	notificationId	Boolean	Markiert als gelesen
getUnread()	userId	Notification[]	Gibt ungelesene Benachrichtigungen

Assoziationen: - N Notification : 1 User (Benachrichtigungen pro User)

Klasse: GameRule

Verantwortlichkeit: Konfigurierbare Spielregeln und XP-Systeme

Attribute:

Attribut	Typ	Beschreibung
easy_xp	Integer	XP für leichte Quests (default: 50)
medium_xp	Integer	XP für mittlere Quests (default: 100)
hard_xp	Integer	XP für schwere Quests (default: 150)
level_threshold	Integer	XP für Level-Up (default: 500)
xp_per_minute_timer	Integer	XP pro Minute Timer-Bonus (default: 2)
max_level	Integer	Maximales Level (default: 50)
updated_at	DateTime	Letzte Regeländerung

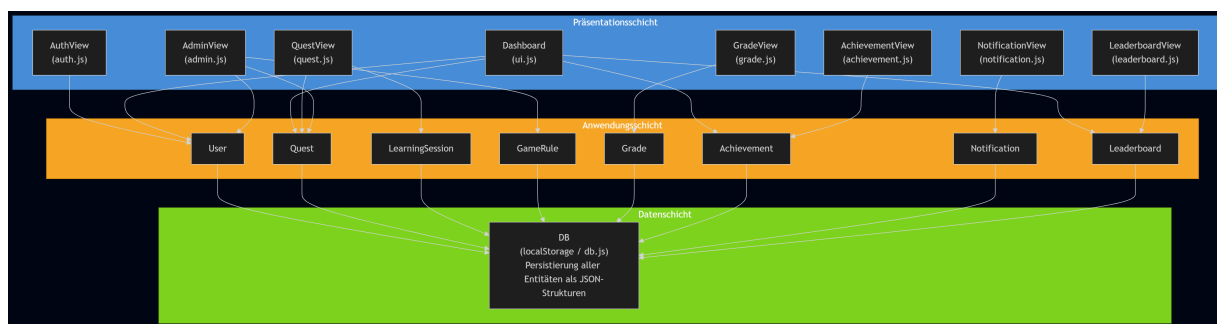
Methoden:

Methode	Parameter	Rückgabe	Beschreibung
<code>updateRules()</code>	{easy_xp, medium_xp, ...}	Boolean	Aktualisiert Spielregeln
<code>getRules()</code>	-	GameRule	Gibt aktuelle Regeln zurück
<code>calculateXPForLevel()</code>	level	Integer	XP-Schwellwert für Level

Assoziationen: - 1 GameRule : N Quest (Alle Quests nutzen die gleichen Regeln)

3. Architekturübersicht (3-Schichten-Modell)

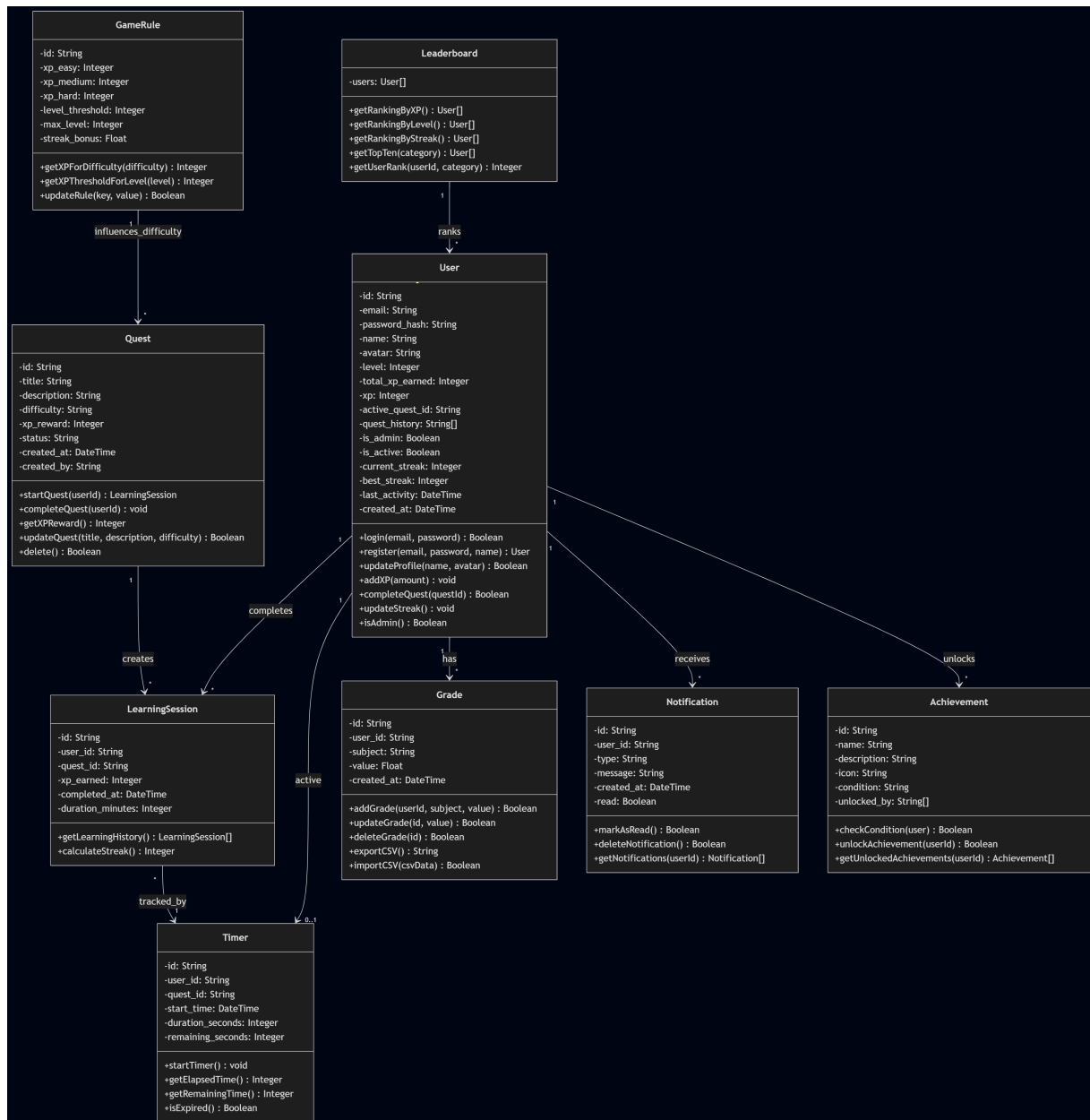
Das StudyQuest-System folgt einer 3-Schichten-Architektur, die eine klare Trennung von Verantwortlichkeiten sicherstellt:



Schicht	Verantwortlichkeit	Komponenten
Präsentation	Benutzerinteraktion, Formularvalidierung, View-Rendering	auth.js, ui.js, quest.js, admin.js, grade.js, leaderboard.js, achievement.js, notification.js + CSS
Anwendung	Business-Logik, XP-Berechnung, Regelvalidierung, Achievement-Prüfung	User, Quest, LearningSession, Grade, Achievement, Leaderboard, Notification, GameRule
Daten	Persistierung (localStorage), CRUD-Operationen, Datenintegrität	DB (db.js)

Kommunikationsfluss: - Präsentation → Anwendung: Benutzeraktionen (z.B. „Quest starten“) rufen Methoden der Anwendungsschicht auf. - Anwendung → Daten: Business-Logik-Klassen persistieren Ergebnisse über die DB-Klasse. - Anwendung → Präsentation: Observer-Pattern benachrichtigt UI-Komponenten über Änderungen (z.B. Level-Up, Achievement).

4. Klassendiagramm-Übersicht



5. Assoziationen und Kardinalitäten

Assoziation	Kardinalität	Beschreibung
User - Quest	1:N	Ein User kann viele Quests abschließen
User - Grade	1:N	Ein User hat viele Schulnoten
User - Achievement	N:M	Ein User hat mehrere Badges, ein Badge mehrere User
User - LearningSession	1:N	Ein User hat viele Sessions
User - Notification	1:N	Ein User empfängt viele Benachrichtigungen
Quest - LearningSession	1:N	Eine Quest hat viele Completion-Sessions
Quest - GameRule	N:1	Viele Quests folgen einer GameRule
Leaderboard - User	N:1	Leaderboard referenziert viele User

7. Design-Muster und Architektur-Überlegungen

Observer-Pattern (UC09, UC12)

- **Verwendung:** Benachrichtigungssystem
- **Komponenten:**
- Subject: QuestSystem (notifiziert Observer bei Quest-Abschluss)
- Observer: NotificationSystem, AchievementSystem, LeaderboardSystem
- **Vorteil:** Lose Kopplung, automatische Updates

Strategy-Pattern (UC14)

- **Verwendung:** Verschiedene XP-Berechnungsstrategien
- **Szenarien:** Easy/Medium/Hard + Timer-Bonus
- **Vorteil:** Leicht erweiterbar für neue Spielregeln

Transaction-Pattern (UC05)

- **Verwendung:** Atomare Quest-Completion
 - **Schritte:** 1. XP berechnen 2. User updaten 3. Session speichern 4. Achievements prüfen
 - **Vorteil:** Verhindert Doppel-XP und Datenkonsistenz
-

8. Anforderungsverfolgung

Use Case	Beteiligte Klassen	Hauptfluss
UC01 - Registrieren & Einloggen	User	User.register() → Email validieren
UC02 - Passwort zurücksetzen	User	User: Token-generierung → Passwort-Hash aktualisieren (Deviation: nicht implementiert)
UC03 - Profil	User	User.updateProfile() → Daten speichern
UC04 - Quest Start	User, Quest	User.setActiveQuest(), Quest.getQuestInfo()
UC05 - Quest Complete	User, Quest, LearningSession, GameRule, Achievement	User.addXP(), LearningSession.logSession(), Achievement.checkUnlock()
UC06 - Timer	LearningSession	LearningSession.applyTimerBonus()
UC07 - Grades	Grade	Grade.calculateAverage(), Grade.exportToCSV()
UC08 - Dashboard	User, Quest, Achievement, Leaderboard	Aggregiert Daten aller Klassen

Use Case	Beteiligte Klassen	Hauptfluss
UC09 - Notifications	Notification, User	Notification.createNotification(), markAsRead()
UC10 - CSV Im/Export	Grade	Grade.importFromCSV(), exportToCSV()
UC11 - Achievements	Achievement, User	Achievement.checkUnlock(), unlockForUser()
UC12 - Leaderboard	Leaderboard, User	Leaderboard.calculateRankings(), updateStreaks()
UC13 - Quest Catalog	Quest, GameRule	Quest CRUD, GameRule.updateRules()
UC14 - Game Rules	GameRule	GameRule.updateRules()
UC15 - User Mgmt	User	User.is_admin, is_active (Admin-Operationen)

9. Datenintegrität und Constraints

Constraint	Beschreibung	Implementierung
Unique Email	Keine doppelten E-Mails	DB-Constraint + App-Validierung
Positive XP	XP kann nicht negativ sein	Validierung in User.addXP()
Level Range	Level 1 bis max_level	GameRule.max_level prüfen
Quest Completion	Jede Quest max. 1x pro User	User.quest_history.includes(questId)
Active Quest Limit	Max. 1 aktive Quest	User.active_quest_id null oder 1 ID
Grade Range	Noten im gültigen Bereich	Grade-Validierung (1.0-6.0 oder 0-100)
Achievement Unlock	Achievement-Bedingung erfüllt	Achievement.checkUnlock()